

Recommendation Engine Using Graph Neural Networks

Submitted by

Retovash Ghosh (13000123070)

Saptarshi Bhunia (13000123081)

Swarnava De Sarkar (13000123118)

Shubhadip Mondal (13000123091)

<Group 03>

Final Year 7th Semester
October, 2026

Submitted for the partial fulfillment for the degree of
Bachelor of Technology in
Computer Science and Engineering



Techno Main Salt Lake,
EM 4/1, SaltLake, Sector V, Kolkata – 700 091

Department of Computer Science and Engineering
Techno Main Salt Lake
Kolkata – 700 091, West Bengal, India.

APPROVAL

This is to certify that the project entitled "Recommendation Engine Using Graph Neural Networks" prepared by Retovash Ghosh (13000123070), Saptarshi Bhunia (13000123081), Swarnava De Sarkar (13000123118) and Shubhadip Mondal (13000123091) be accepted in partial fulfillment for the degree of Bachelor of Technology in Computer Science and Engineering.

It is to be understood that by this approval, the undersigned does not necessarily endorse or approve any statement made, opinion expressed or conclusion drawn thereof, but approves the report only for the purpose for which it has been submitted.

(Signature of the Internal Guide)

(Signature of the HOD)

(Signature of the External Examiner)

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide in the department of Computer Science and Engineering. We are extremely thankful for the keen interest our guide took in advising us, for the books, reference materials and support extended to us.

Last but not the least we convey our gratitude to all the teachers for providing us the technical skill that will always remain as our asset and to all non-teaching staffs for the gracious hospitality they offered us.

Place: Techno Main Salt Lake

Date:

.....

.....

.....

.....

CONTENTS

1 Introduction	5
2 Related Studies	6
3 Problem Definition and Preliminaries	7
4 Proposed Solution	7
5 Project Planning	8
6 Requirement Analysis	9
6.1 Requirement Matrix	9
6.2 Requirement Elaboration	10
7 Design	10
7.1 Technical Environment	10
7.2 Detailed Design	11
8 Implementation	12
9 Test Plans, Results and Analysis	12
10 Conclusion	13
11 References	14
12 APPENDIX A – NPTEL Certificate	14

1 Introduction

Project Purpose and Topic

Online shoppers face thousands of products and rarely have time to browse them all. Recommendation systems help by surfacing relevant items, but traditional approaches such as popularity ranking or simple collaborative filtering treat each user and product largely in isolation, missing higher-order relationships hidden in shopping behavior. This project builds an end-to-end recommendation engine for an e-commerce platform using Graph Neural Networks (GNNs). By modelling user interactions as a user-product graph, the system learns vector embeddings to drive a retrieval, filtering, ranking, and diversification pipeline, returning personalized recommendations through a standalone FastAPI service.

Technical and Business Domain

- **Business Domain:** E-Commerce, Personalized Marketing, Customer Experience Enhancement.
- **Technical Domain:**
 - Hardware: Standard multi-core CPU with at least 16GB RAM; NVIDIA GPU (CUDA-enabled) recommended for model training.
 - Operating System: Linux / Ubuntu (Recommended for deployment), Windows/macOS (Development).
 - Software/Frameworks: Python, PyTorch, PyTorch Geometric, ChromaDB, FastAPI, PostgreSQL, Redis, Docker, MLflow.

Table 1: Glossary and Keywords

Keyword / Term	Definition
GNN	Graph Neural Network, a class of neural networks designed to process data represented by graph data structures.
LightGCN	A simplified graph convolution model optimized for collaborative filtering.

BPR	Bayesian Personalized Ranking, a loss function that encourages observed interactions to score higher than unobserved ones.
ChromaDB	A vector database used to store embeddings and perform efficient similarity search for recommendations.
Cold Start	The challenge of providing recommendations for new users or new products with little to no interaction history.
NDCG	Normalized Discounted Cumulative Gain, a measure of ranking quality.

2 Related Studies

Traditional recommendation systems heavily rely on matrix factorization and basic collaborative filtering, which only learn from direct interactions and do not explicitly exploit multi-hop neighborhood structures [1]. Recent advancements in deep learning on graphs have introduced methods like Inductive Representation Learning on Large Graphs (GraphSAGE) [4] and Graph Attention Networks (GAT) [5], which capture complex relational dynamics.

For e-commerce recommendation specifically, Neural Graph Collaborative Filtering (NGCF) integrates the user-item bipartite graph directly into the embedding process [3]. However, NGCF's heavy non-linear transformations were found to be overly complex for collaborative filtering. This project leverages LightGCN [1], which simplifies the architecture by removing feature transformations and non-linear activations, keeping only neighbourhood aggregation. The model is optimized using Bayesian Personalized Ranking (BPR) [2] from implicit feedback. For fast real-time serving of these vector embeddings, billion-scale similarity search architectures like ChromaDB have become the industry standard.

3 Problem Definition and Preliminaries

Problem Statement:

To design and implement a scalable recommendation engine that learns user preferences from graph-structured shopping behavior and serves accurate, diverse, and timely product recommendations, including reasonable handling of new users and new products, through a service that any e-commerce application can call.

Scope and Boundaries:

- **In Scope:** Building a time-based data pipeline, constructing a weighted user-product graph, training a LightGCN model for embeddings, implementing a ChromaDB-based retrieval system, and exposing a FastAPI serving endpoint with Redis caching.
- **Exclusions:** Real-time continuous online training (models will be batch-trained), and direct multimodal processing (text/image features) via GNNs (this is slated for future scope, with cold starts currently handled by metadata fallbacks).

Methodology and Preliminaries:

The project relies on implicit feedback where user actions (views, clicks, purchases) signify preference. We map this to a bipartite graph structure. Vector representations for users and nodes are generated using PyTorch and PyTorch Geometric. Similarity mapping is handled by ChromaDB, utilizing nearest-neighbor searches on the generated embeddings.

4 Proposed Solution

We propose an end-to-end recommendation pipeline centred around a LightGCN model. Shopping activity is converted into a bipartite graph where nodes are users and products, and edges are interactions.

Specific Contributions and Workflow:

1. Signal Weighting: Different interactions carry different weights (e.g., View = 1, Click = 2, Wishlist = 3, Add to Cart = 4, Purchase = 5).

2. **Embedding Generation:** LightGCN captures multi-hop user-product relationships through neighbourhood aggregation.
3. **Loss Optimization:** The model trains using BPR loss, ensuring items a user interacted with are ranked higher than unobserved items.
4. **Retrieval and Serving Component:** Embeddings are stored in ChromaDB. When a request hits the API, the system queries the user embedding against product embeddings, retrieves the most similar items, applies availability filters, and enforces diversification rules before returning recommendations.
5. **Cold Start Component:** New shoppers receive recommendations based on popular products and immediate session signals. New products are retrieved via a metadata-based fallback.

5 Project Planning

The project follows an Agile/Iterative Software Development Life Cycle (SDLC), allowing for continuous refinement of the model architecture and API endpoints.

Table 2: Project Roadmap and Milestones

Phase	Tasks / Deliverables	Status	Assigned To
1. Foundation	Event pipeline, temporal splits, and baseline models	Completed	Data/Graph Engineer
2. Learning	User-product graph construction, LightGCN training	In Progress	GNN/ML Engineer
3. Discovery	Vector retrieval (ChromaDB), filtering, and ranking	In Progress	Retrieval Engineer
4. Serving	FastAPI integration and Redis caching	In Progress	Integration Engineer

5. Validation	Offline evaluation (Recall, NDCG) against baselines	In Progress	Integration Engineer
6. Production	Versioned model bundles (MLflow), Dockerization	Planned	Whole Team

Cost Analysis: Software development relies entirely on open-source tools. Anticipated costs are limited to cloud compute resources (AWS/GCP) for GPU-based training and hosting, which will be managed using student credits or local computing where possible.

6 Requirement Analysis

6.1 Requirement Matrix

Table 3: Requirement Matrix

Req ID	Requirement Area	Description	Priority
REQ-DAT-01	Data Pipeline	System must process raw user events into a weighted bipartite graph.	High
REQ-MOD-01	GNN Model	System must generate user/product embeddings using LightGCN.	High
REQ-RET-01	Retrieval	System must retrieve top-K candidates using vector similarity.	High
REQ-API-01	API Serving	System must expose a GET /recommendations/{user_id} endpoint.	High
REQ-FIL-01	Business Logic	System must filter out-of-stock items.	Medium

6.2 Requirement Elaboration

6.2.1 Data Pipeline (REQ-DAT-01)

The system needs a clean, leakage-safe pipeline. It must parse historical shopping events and map them to standard weights. The data split must be strictly temporal (e.g., train on months 1-3, validate on month 4, test on month 5) to prevent future-data leakage.

6.2.2 GNN Model (REQ-MOD-01)

The core intelligence requirement. The system needs to instantiate a LightGCN model via PyTorch. It must successfully map the graph data into dense vector embeddings and continuously track experiments (loss, epochs) using MLflow.

6.2.3. Retrieval System (REQ-RET-01)

The system must retrieve the top-K candidate products using vector similarity search on embeddings stored in ChromaDB. It should return the most relevant items based on similarity scores while supporting filtering and low-latency retrieval.

6.2.4 API Serving (REQ-API-01)

Any front-end e-commerce application must be able to query the engine. The FastAPI endpoint must accept a user ID and a k parameter, returning a JSON array of k product IDs sorted by relevance score, with latency under 200ms.

7 Design

7.1 Technical Environment

Table 4: Technical Environment and Configuration

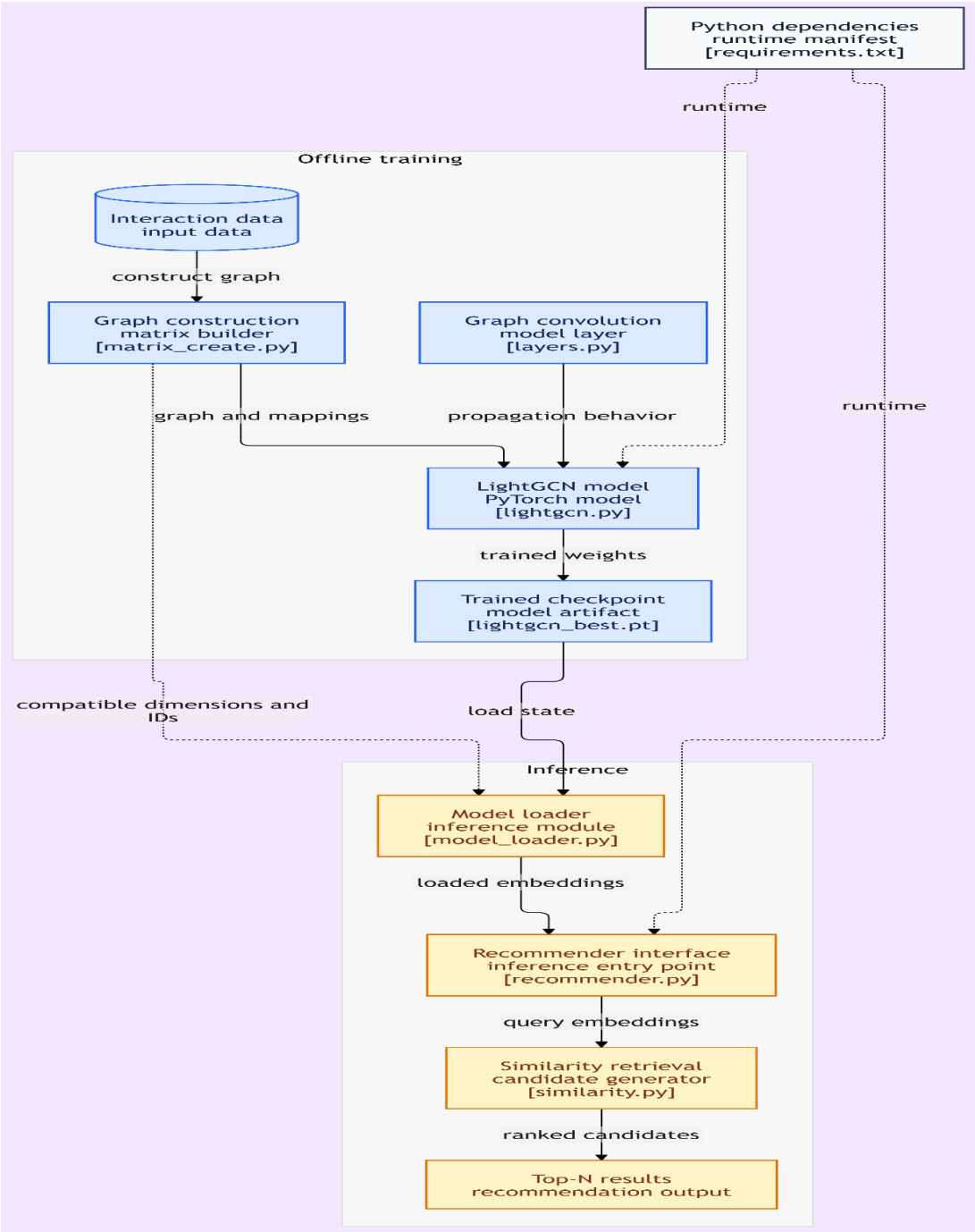
Component	Minimum Configuration / Tool
Processing / Logic	Python 3.9+, Pandas, NumPy
Machine Learning	PyTorch, PyTorch Geometric, MLflow

Retrieval Engine	ChromaDB (Vector DB for Similarity Search)
Web Service API	FastAPI, Uvicorn
Databases	PostgreSQL (Metadata), Redis (Cache)
Deployment	Docker

7.2 Detailed Design

7.2.1 System Architecture Pipeline

The engine operates as a sequential pipeline, ensuring data flows logically from raw logs



to serve predictions.

7.2.2 Processing and Retrieval Modules

1. Prepare & Connect: Cleans events and builds the weighted bipartite graph.
2. Learn: The ML module where LightGCN outputs standard vector embeddings.
3. Retrieve & Filter: ChromaDB retrieves the most similar items, while filtering logic applies stock availability rules.
4. Rank & Diversify: Contextual signals boost or penalize candidate scores. A diversification algorithm ensures the final Top-N list isn't entirely homogeneous.

8 Implementation

The focus of the current deliverables encompasses analysis, design, and initial prototyping. Currently, Phase 1 (Foundation) and Phase 2 (Learning) are under active development. The data preprocessing modules, graph construction logic, and initial LightGCN architectures have been drafted. The bipartite graph correctly assigns varying weights based on interaction depth (View=1 to Purchase=5). Ongoing work involves tuning the BPR loss and validating the generated embeddings against simple baseline models (popularity-based and item-similarity matrices).

9 Test Plans, Results and Analysis

Performance Metrics Defined:

- Recall@K: Measures the proportion of relevant items found in the top-K recommendations.
- Precision@K: Measures the proportion of the top-K recommendations that are actually relevant.
- NDCG@K: Normalized Discounted Cumulative Gain; evaluates ranking quality by penalizing relevant items that appear lower in the top-K list.
- Hit Rate@K: The fraction of users for whom at least one relevant item is present in the top-K list.

Table 5: Test Plans

Test Case ID	Description	Input	Expected Output	Status
MOD-01	Validate model training loss	Prepared BPR graph data	BPR loss strictly decreases over epochs	Completed
RET-01	ChromaDB Retrieval output	User Vector	Returns K closest Product Vectors	Completed
API-01	Endpoint response time	GET /recommendations/123?k=20	JSON list of 20 items in <200ms	Pending
FIL-01	Stock filtering	Top-K containing out-of-stock item	Out-of-stock item is removed from final list	Completed

Analysis of test results will be populated in subsequent phases upon offline evaluation completion, comparing the LightGCN output to popularity and basic matrix-factorization baselines.

10 Conclusion

Project Benefits:

This project delivers a modular, scalable recommendation engine that easily integrates into any e-commerce application via a simple API. By moving beyond traditional isolation models and utilizing graph-based LightGCN embeddings, the system uncovers deep, multi-hop user-product relationships. This results in recommendations that are

highly personalized, relevant, and varied, directly offering better discovery than standard popularity-based suggestions.

Future Scope:

- Implementation of alternative GNN architectures such as GraphSAGE and GAT, accommodating heterogeneous graphs.
- Feature-aware and multimodal models utilizing product text descriptions and image metadata to natively handle cold-start products.
- Session-based personalization for real-time preference updates during an active shopping session.
- Online A/B testing, live metrics monitoring, and automated retraining pipelines in production.

11 References

- [1] X. He, K. Deng, X. Wang, Y. Li, Y. Zhang and M. Wang, "LightGCN: Simplifying and Powering Graph Convolution Network for Recommendation," SIGIR, 2020.
- [2] S. Rendle, C. Freudenthaler, Z. Gantner and L. Schmidt-Thieme, "BPR: Bayesian Personalized Ranking from Implicit Feedback," UAI, 2009.
- [3] X. Wang, X. He, M. Wang, F. Feng and T.-S. Chua, "Neural Graph Collaborative Filtering," SIGIR, 2019.
- [4] W. Hamilton, Z. Ying and J. Leskovec, "Inductive Representation Learning on Large Graphs," NeurIPS, 2017.
- [5] P. Velickovic et al., "Graph Attention Networks," ICLR, 2018.
- [6] J. Johnson, M. Douze and H. Jegou, "Billion-scale similarity search with GPUs," IEEE Transactions on Big Data, 2019.

12 APPENDIX A – NPTEL Certificate

[Insert Digital Copy of NPTEL Course Certificate Here]